

ViewEngine及Helpers

- 6-1: View Engine

View Engine (檢視引擎) 簡介	6-2
Razor 檢視引擎的優點	6-3
View Engine的用途	6-4
Razor語法	6-6

- 6-2:HTML Helpers

HTML Helpers	6-9
一般的Helpers	6-11
Form表單的HTML Helpers	6-13

- 6-3:Tag Helpers

Tag Helpers	6-20
常用 Tag Helpers 清單與用途	6-23
HTML Helpers 對應的 Tag Helpers	6-25
HTML Helpers vs Tag Helpers 比較表	6-26
練習-Form表單	6-27

Module 06

View Engine (檢視引擎) 簡介

- 用來將 **Server** 端資料 與 **HTML** 樣板 結合，產生最終回傳給瀏覽器的網頁內容 (**HTML**) 。
- **ASP.NET MVC** 常見的 **View Engine** :
 - **ASPX View Engine (Original ASPX)** : 語法與傳統 **WebForms** 類似，較偏舊式寫法。
 - **Razor View Engine** : 語法更簡潔，使用 **@** 來嵌入 **C#**，可讀性與維護性較佳。
- 在 **ASP.NET Core** 中，主要預設使用 **Razor** 為檢視引擎 (**ASPX** 已非主流) 。

Razor 檢視引擎的優點

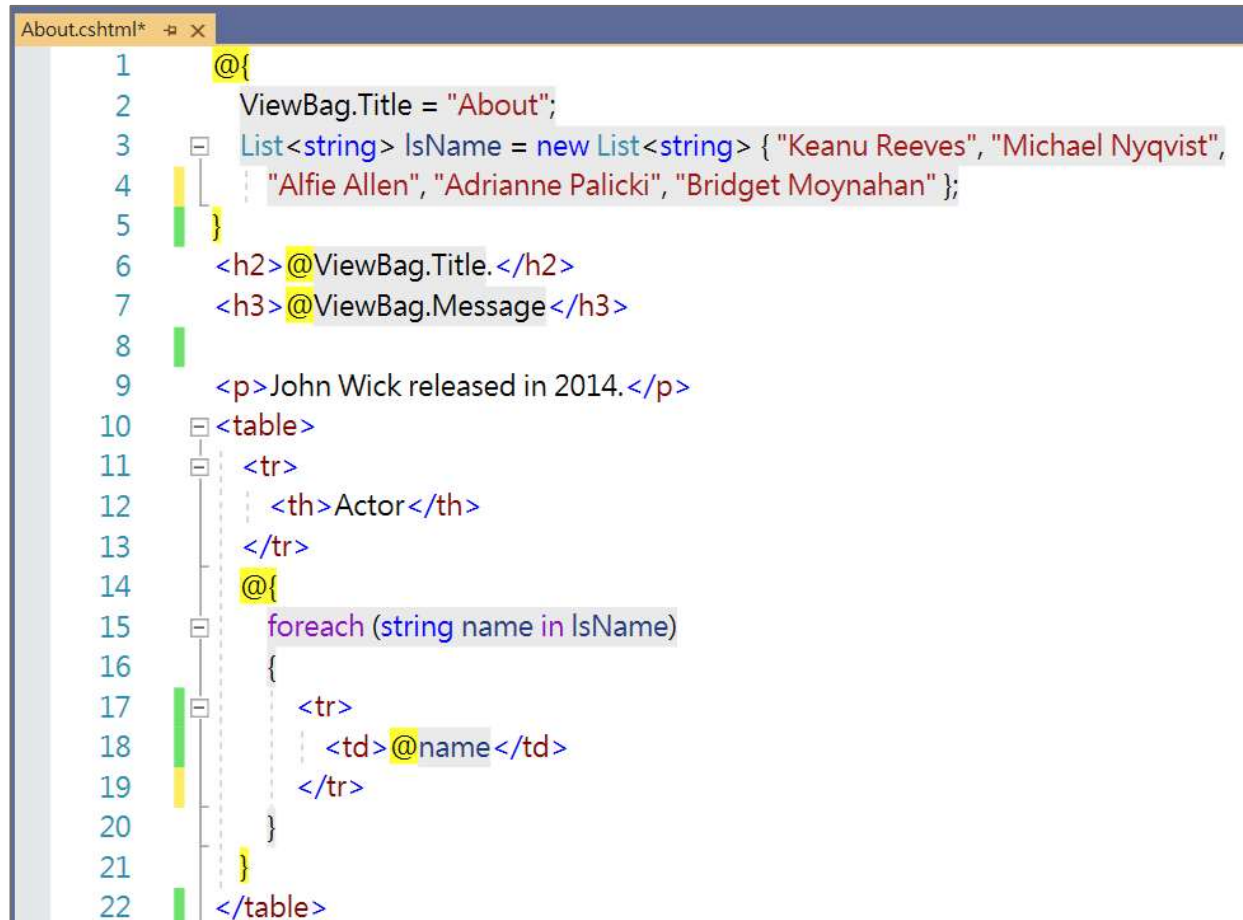
- 語法精簡、可讀性高：使用 @ 進行 C# 與 HTML 的整合，結構清楚。
- 容易上手：相較傳統 ASPX 語法更直覺，學習曲線較平緩。
- 編輯器友善：.cshtml 為純文字檔，可使用各種文字編輯器或 IDE 編輯。
- 良好的 IntelliSense 支援：在 Visual Studio / VS Code 中可提供語法提示與自動完成。
- 前後端整合性佳：可在 View 中同時撰寫 HTML、CSS、JavaScript，並嵌入必要的後端資料輸出與邏輯。
- Razor 讓「畫面模板」與「資料輸出」結合得自然又乾淨。

View Engine 的用途 -1

- **View Engine 負責產生最終的 HTML。**
 - 將伺服器端資料套入檢視樣板，輸出原始 HTML 文件，用於呈現 Web 應用程式的使用者介面（UI）。
- **ASP.NET 可使用 Razor + HTML 建立 View。**
 - 在檢視頁面中可同時撰寫 HTML，並使用 Razor 語法輸出資料與嵌入 C# 程式碼。
- **Razor 檢視檔副檔名為 .cshtml**
 - .cshtml 是 ASP.NET 的 Razor View 檔案格式，用於製作檢視頁面（View）。

View Engine 的用途 -2

- View Engine 的工作就是把「資料 + 樣板」組裝成瀏覽器看得懂的 HTML。



```
1  @{
2      ViewBag.Title = "About";
3      List<string> lsName = new List<string> { "Keanu Reeves", "Michael Nyqvist",
4          "Alfie Allen", "Adrianne Palicki", "Bridget Moynahan" };
5  }
6  <h2>@ViewBag.Title.</h2>
7  <h3>@ViewBag.Message</h3>
8
9  <p>John Wick released in 2014.</p>
10 <table>
11     <tr>
12         <th>Actor</th>
13     </tr>
14     @{
15         foreach (string name in lsName)
16         {
17             <tr>
18                 <td>@name</td>
19             </tr>
20         }
21     }
22 </table>
```

Razor 語法 -1

- Razor文件使用 @ 來區分後端程式C#與HTML標籤。

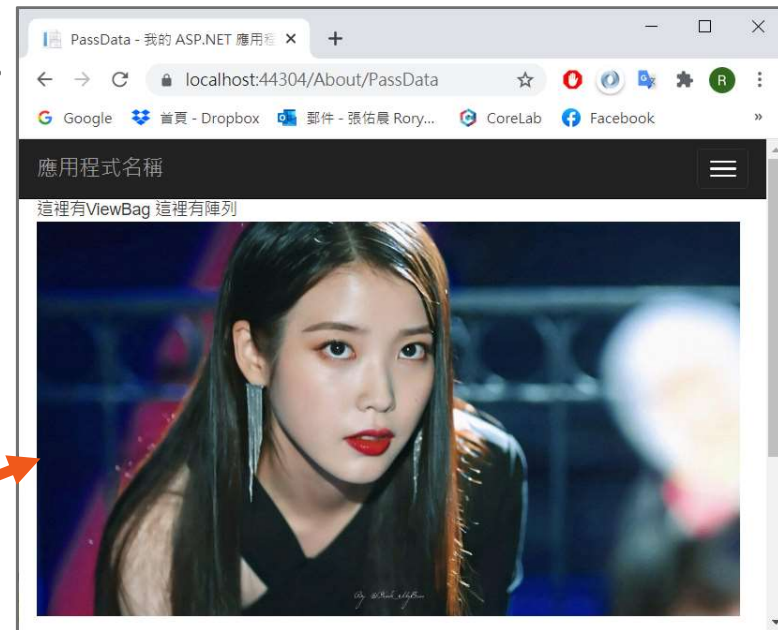
```
<p>現在時間：@DateTime.Now.ToString()</p>
```

- 使用 @{ } 表示後端程式C#的程式碼區塊。
- 使用 @: 或 <text></text> 在C#的的程式碼區塊中顯示文字。
- 使用 @() 區分C#與HTML的內容。

```
@{  
    ViewBag.Title = "PassData";  
    @: 這裡有ViewBag  
    int imgID = 2;  
    string[] Fruits = { "Apple", "Banana", "Coconut" };  
    <text>這裡有陣列</text>  
}  
<br>  

```

1-6



右晨 編

Razor語法 -2

- 使用 @* *@ 註解。

```
@*這是註解*@
```

- 使用 @@ 顯示一個@符號在網頁上。

```
<p>它本來不是小老鼠也不是at，「@@」符號到底哪來的？</p>
```

Razor 語法 -3

- 在Razor文件使用判斷式(if、switch)及迴圈(for、while、foreach)。

```
@{
    string[] Fruits = { "Apple", "Banana", "Coconut" };
}

@if(Fruits != null)
{
    <table class="table table-hover">
        <thead><tr><th>ID</th><th>水果</th></tr></thead>
        <tbody>
            @for(int i = 0; i < Fruits.Length; i++)
            {
                <tr><td>@i</td><td>@Fruits[i]</td></tr>
            }
        </tbody>
    </table>
}
else
{
    <h3>沒有水果了!!!</h3>
}
```



ID	水果
0	Apple
1	Banana
2	Coconut

HTML Helpers -1

- **HTML Helpers 是 ASP.NET MVC / ASP.NET Core 提供的一組輔助方法，可在 Razor View (.cshtml) 中使用 @Html 產生常見的 HTML 元素。**
- **主要目的：**
 - 減少重複撰寫 HTML 程式碼
 - 協助資料繫結 (**Model Binding**)
 - 提升表單與畫面的開發效率

```
@Html.TextBox("Name")  
@Html.TextBoxFor(m => m.Name)  
@Html.LabelFor(m => m.Email)
```

HTML Helpers -2

- 與 **Model** 結合，支援資料繫結
- 自動產生正確的 **name**、**id**
- 可搭配 **Model Validation**
- 適用於表單、輸入欄位、標籤等常見 **UI** 元素
- 常見應用場景：
 - 表單輸入欄位 (**Input**)
 - **Label** 與 **Validation Message**
 - 表單送出 (**Form**)
 - **Partial View** 呼叫

```
@model CoreMVCLab.Models.Member

@{
    ViewData["Title"] = "Index";
}

<h1>Index</h1>

<div>
    <h4>Member</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.BirthDate)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.BirthDate)
        </dd>
    </dl>
</div>
```

一般的Helpers -1

- **Helpers**為使用在Razor文件中的方法，可產生HTML標籤。
- **Html.Raw()**：確保輸出的值是HTML標籤。

```
@{  
    string str5der = "<h2>我大意了 沒有閃</h2>";  
}  
@Html.Raw(str5der)  
@str5der
```

我大意了 沒有閃
<h2>我大意了 沒有閃</h2>

- **Html.ActionLink()**：產生HTML連結標籤。

```
@Html.ActionLink("關於", "Index", "About")
```

關於

一般的Helpers -2

- **Url.Action()**：產生指定的Action與Controller路徑。

```
@Url.Action("stringMethod", "About")
```

- **Url.Content()**：產生於Server上URL絕對網址路徑的方法。

```

```



```

```

- **Html.ActionLink()**與**Url.Action()**可使用Route傳遞參數。
- 在網站部屬時，Server站台與專案開發時的路徑可能不同，建議專案中多使用**Url.Action()**、**Url.Content()**，產生符合Server站台設定環境的連結或網址。

Form表單的HTML Helpers -1

- **BeginForm**
- **BeginRouteForm**
- **EndForm**
- **CheckBox**
- **DropDownList**
- **ListBox**
- **Hidden**
- **Password**
- **RadioButton**
- **TextArea**
- **TextBox**
- **Label**
- **DisplayName**
- **DisplayText**
- **Display**
- **Editor**

Form表單的HTML Helpers -2

- **Html.BeginForm()**、**Html.EndForm()**：產生Form表單HTML標籤，常搭配 **using(){ }** 一起使用（無須使用**Html.EndForm()**結尾）。

```
@using (Html.BeginForm("FormHelpers", "About", FormMethod.Post  
, new { enctype="application/x-www-form-urlencoded" })))  
{  
    ...  
}
```



```
<form action="/About/FormHelpers" enctype="application/x-www-form-urlencoded" method="post">  
    ...  
</form>
```

Form表單的HTML Helpers –3

- **Html.TextBox()**：產生Input HTML標籤(type="text")。

```
@Html.TextBox("MemberName", "Guest", new { placeholder = "請輸入會員姓名" })
```



```
<input id="MemberName" name="MemberName" placeholder="請輸入會員姓名" type="text" value="Guest">
```

- **Html.Label()**：產生Label標籤。

```
@Html.Label("MemberName", "會員姓名：")
```



```
<label for="MemberName">會員姓名：</label>
```

- **Html.Password()**：產生Input HTML標籤(type="password")。

```
@Html.Password("MemberPW", null, new { placeholder = "請輸入會員密碼" })
```



```
<input id="MemberPW" name="MemberPW" placeholder="請輸入會員密碼" type="password">
```


Form表單的HTML Helpers -4

- **Html.TextArea()** : 產生TextArea標籤。

```
@Html.TextArea("Descript", null, 10, 20, new { placeholder = "描述", @class = "form-control" })
```

↓

```
<textarea class="form-control" cols="20" id="Descript" name="Descript"  
placeholder="描述" rows="10"></textarea>
```

- **Html.Hidden()** : 產生Input HTML標籤(type="hidden")。

```
@Html.Hidden("MemberID") → <input id="MemberID" name="MemberID" type="hidden" value="">
```

- **Html.CheckBox()** : 產生Input HTML標籤(type="checkbox")。

```
@Html.CheckBox("MemberHobby", false, new { value = "music" }) @:音樂  
@Html.CheckBox("MemberHobby", false, new { value = "movie" }) @:電影
```

↓

```
<input id="MemberHobby" name="MemberHobby" type="checkbox" value="music"> 音樂  
<input id="MemberHobby" name="MemberHobby" type="checkbox" value="movie"> 電影
```


Form 表單的HTML Helpers –5

- **Html.RadioButton()**：產生Input HTML標籤(type="radio")。

```
@Html.RadioButton("MaritalStatus", "married", false, new { @class = "form-check-input" }) @:已婚  
@Html.RadioButton("MaritalStatus", "unmarried", true, new { @class = "form-check-input" }) @:未婚
```



```
<input class="form-check-input" id="MaritalStatus" name="MaritalStatus"  
      type="radio" value="married">已婚  
<input checked="checked" class="form-check-input" id="MaritalStatus" name="MaritalStatus"  
      type="radio" value="unmarried">未婚
```

Form 表單的HTML Helpers –6

- **Html.DropDownList()** : 產生Select Options標籤。

```
@Html.DropDownList("City", new List<SelectListItem>() {  
    new SelectListItem() { Text="Taipei", Value="0" },  
    new SelectListItem() { Text="New York", Value="1" },  
    new SelectListItem() { Text="Tokyo", Value="2" } }, "請選擇城市", new { @class = "form-select" })
```



```
<select class="form-select" id="City" name="City">  
    <option value="">請選擇城市</option>  
    <option value="0">Taipei</option>  
    <option value="1">New York</option>  
    <option value="2">Tokyo</option>  
</select>
```

Form 表單的HTML Helpers –7

- **Html.DropDownList()** : 使用ViewBag帶入資料。

— Controller :

```
public ActionResult FormHelpers()
{
    StoreDBEntities db = new StoreDBEntities();
    ViewBag.allCity = new SelectList(db.City, "CityID", "CityName");
    return View();
}
```

— View :

```
@Html.DropDownList("City", (SelectList)ViewBag.allCity, "請選擇城市", new { @class = "form-select" })
```

Tag Helpers -1

- 是 ASP.NET Core 提供的功能，可在 Razor View (.cshtml) 中，使用 接近原生 HTML 的語法 來產生動態內容與資料繫結。
- Tag Helpers 會在伺服器端轉換成標準 HTML，瀏覽器端只會接收到純 HTML。
- Tag Helpers 的特色：
 - 使用 HTML 標籤 + asp-* 屬性
 - 語法直覺，前端與後端都容易閱讀
 - 與 Model Binding、Validation、Routing 深度整合
 - 擁有良好的 IntelliSense 支援
 - 為 ASP.NET Core 官方推薦 的 View 撰寫方式

```
@model CoreMVCLab.Models.Member

@{
    ViewData["Title"] = "TagHelpers";
}

<h1>TagHelpers</h1>

<div>
    <h4>Member</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            <label asp-for="Name" class="control-label"></label>
        </dt>
        <dd class="col-sm-10">
            <span>@Model.Name</span>
        </dd>
        <dt class="col-sm-2">
            <label asp-for="BirthDate" class="control-label"></label>
        </dt>
        <dd class="col-sm-10">
            <span>@Model.BirthDate</span>
        </dd>
    </dl>
</div>
```

Tag Helpers -2

- 常見 Tag Helpers 類型：
 - 表單相關：asp-for、asp-validation-for
 - 路由相關：asp-action、asp-controller、asp-route-*
 - 連結產生：asp-page、asp-area
 - Partial View：partial
- 啟用 Tag Helpers，在 Razor View 中加入（通常已預設）：

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```
- Tag Helpers 讓「寫 View」更像在寫 HTML，而不是寫 C#。

Tag Helpers -3

```
<form asp-action="Create" asp-controller="Member" method="post">
  <label asp-for="Name"> </label>
  <input type="text" asp-for="Name" />
  <label asp-for="BirthDate"> </label>
  <input type="date" asp-for="BirthDate" />
  <textarea asp-for="Remark"> </textarea>
  <input type="hidden" asp-for="Id" />
  @foreach (var opt in Model.InfoSource)
  {
    <label>
      <input type="checkbox"
        name="InfoSource"
        value="@opt" />
      @opt
    </label>
  }
  @foreach (var opt in Model.Gender)
  {
    <label>
      <input type="radio" asp-for="Gender" value="@opt"/> @opt
    </label>
  }
  <select class="form-select" asp-for="Select" asp-items="Model.SelectList"> </select>
</form>
```

常用 Tag Helpers 清單與用途 -1

- 表單與資料繫結：
 - **asp-for**：與 Model 屬性繫結 (input/select/textarea/label)
 - **asp-validation-for**：顯示指定欄位的驗證訊息
 - **asp-validation-summary**：顯示整體驗證摘要 (ModelState)
- 路由與連結：
 - **asp-action**：指定目標 Action
 - **asp-controller**：指定目標 Controller
 - **asp-route-***：附加路由參數 (如 **asp-route-id="10"**)
 - **asp-area**：指定 Area (有分區時用)

常用 Tag Helpers 清單與用途 -2

- 下拉選單：
 - `asp-items`：提供 `<select>` 的選項清單 (`IEnumerable<SelectListItem>`)
- Partial 與元件：
 - `<partial name="...">`：載入 Partial View
 - `asp-page` / `asp-page-handler`：Razor Pages 專用 (非 MVC 主線可略)
- 靜態資源版本控管
 - `asp-append-version="true"`：產生 cache busting (CSS/JS更新不怕快取)

HTML Helpers 對應的 Tag Helpers

HTML Helper	對應的 Tag Helper 寫法
Html.BeginForm() / Html.EndForm()	<form asp-action="Action" asp-controller="Controller" method="post">
Html.TextBox()	<input type="text" asp-for="Property" />
Html.Label()	<label asp-for="Property"></label>
Html.Password()	<input type="password" asp-for="Property" />
Html.TextArea()	<textarea asp-for="Property"></textarea>
Html.Hidden()	<input type="hidden" asp-for="Property" />
Html.CheckBox()	<input type="checkbox" asp-for="Property" />
Html.RadioButton()	<input type="radio" asp-for="Property" />
Html.DropDownList()	<select asp-for="Property" asp-items="Model.SelectList"> </select>

HTML Helpers vs Tag Helpers 比較表

項目	HTML Helpers	Tag Helpers
使用方式	@Html.* 方法呼叫	HTML 標籤 + asp-* 屬性
撰寫風格	偏 C# 語法	接近原生 HTML
可讀性	對後端熟悉者較佳	前端友善、可讀性高
與 Model 繫結	支援	支援
IntelliSense	有	更完整 (HTML + 屬性提示)
前端設計師協作	較不友善	非常友善
學習曲線	對 MVC 舊用戶熟悉	新手較容易上手
ASP.NET Core 建議	可用	官方主流推薦

練習-Form表單

- 請使用HTML Helpers與Tag Helpers做出以下表單。

FormHelpers

會員姓名：

會員密碼：

會員描述：

描述

興趣：☐ 音樂 ☐ 電影

婚姻狀況：☐ 已婚 ☒ 未婚

城市：